Recursion and Merge-Sort Algorithms

Task 1: Understanding the recursion algorithm and testing it for simple and complex cases (Fibonacci sequence and factorial computation)

1a

Extract the relevant recursion sections from recursion_mergesort.py to understand how simple recursion works for different test cases - simple recursion, Fibonacci sequence and the Factorial computation test cases.

```
The sum of the first 5 natural numbers is 15
The 7th Fibonacci number is 13
The factorial of 5 is 120
```

1b

Convert the python code for the three test cases in (1) to iteration (iterative loop) implementation instead of recursion.

```
# ITERATIVE VERSIONS

def sum_iterative(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total


def fibonacci_iterative(n):
    if n <= 0:
        return 0
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

```
def factorial_iterative(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

1c

```
import timeit

# ----------------------------
# RECURSIVE FUNCTIONS
# ----------------------------

def sum_recursive(n):
   if n == 1:
       return 1
   return n + sum_recursive(n - 1)


def fibonacci_recursive(n):
   if n <= 0:
       return 0
   elif n == 1:
       return 1
   return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)


def factorial_recursive(n):
   if n == 0:
       return 1
   return n * factorial_recursive(n - 1)


# ----------------------------
# ITERATIVE FUNCTIONS
# ----------------------------

def sum_iterative(n):
   total = 0
```

```python
    for i in range(1, n + 1):
        total += i
    return total


def fibonacci_iterative(n):
    if n <= 0:
        return 0
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a


def factorial_iterative(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result


# ------------------------------
# BENCHMARKING
# ------------------------------

def benchmark():
    test_values = [10, 20, 30]  # different data sizes

    for n in test_values:
        print(f"\n===== Testing n = {n} =====")

        # SUM
        sum_rec_time = timeit.timeit(lambda: sum_recursive(n), number=1000)
        sum_itr_time = timeit.timeit(lambda: sum_iterative(n), number=1000)

        print(f"Sum Recursive:  {sum_rec_time:.6f}")
        print(f"Sum Iterative:  {sum_itr_time:.6f}")

        # FACTORIAL
```

```
        fact_rec_time = timeit.timeit(lambda: factorial_recursive(n),
number=1000)
        fact_itr_time = timeit.timeit(lambda: factorial_iterative(n),
number=1000)

        print(f"Factorial Recursive: {fact_rec_time:.6f}")
        print(f"Factorial Iterative: {fact_itr_time:.6f}")

        # FIBONACCI
        fib_rec_time = timeit.timeit(lambda: fibonacci_recursive(n),
number=10)
        fib_itr_time = timeit.timeit(lambda: fibonacci_iterative(n),
number=1000)

        print(f"Fibonacci Recursive: {fib_rec_time:.6f}")
        print(f"Fibonacci Iterative: {fib_itr_time:.6f}")


# ----------------------------
# MAIN
# ----------------------------

if __name__ == "__main__":
    benchmark()
```

```
===== Testing n = 10 =====
Sum Recursive:  0.000360
Sum Iterative:  0.000267
Factorial Recursive: 0.000417
Factorial Iterative: 0.000299
Fibonacci Recursive: 0.000096
Fibonacci Iterative: 0.000242

===== Testing n = 20 =====
Sum Recursive:  0.000723
Sum Iterative:  0.000374
Factorial Recursive: 0.001115
Factorial Iterative: 0.000706
Fibonacci Recursive: 0.010834
Fibonacci Iterative: 0.000749
```

```
===== Testing n = 30 =====
Sum Recursive:  0.001315
Sum Iterative:  0.000714
Factorial Recursive: 0.001823
Factorial Iterative: 0.001505
Fibonacci Recursive: 1.406699
Fibonacci Iterative: 0.000871
```

Task 2

2a

```
import random


def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)


def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
```

```
    return result


# TESTING
sizes = [10, 100, 1000]

for size in sizes:
    data = [random.randint(1, 1000) for _ in range(size)]
    sorted_data = merge_sort(data)

    print(f"\nSize: {size}")
    print("First 10 sorted elements:", sorted_data[:10])
_
Size: 10
First 10 sorted elements: [39, 66, 298, 387, 605, 829, 870, 875, 973, 995]

Size: 100
First 10 sorted elements: [9, 12, 15, 43, 52, 78, 79, 89, 98, 102]

Size: 1000
First 10 sorted elements: [1, 1, 2, 2, 4, 4, 5, 5, 5, 6]
```

2b

```
import random


def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

    return arr
```

```
# TESTING
sizes = [10, 100, 1000]

for size in sizes:
    data = [random.randint(1, 1000) for _ in range(size)]
    sorted_data = insertion_sort(data)

    print(f"\nSize: {size}")
    print("First 10 sorted elements:", sorted_data[:10])
```

```
Size: 10
First 10 sorted elements: [125, 289, 314, 320, 384, 489, 532, 563, 813,
994]

Size: 100
First 10 sorted elements: [2, 10, 14, 34, 59, 78, 78, 84, 87, 129]

Size: 1000
First 10 sorted elements: [1, 2, 3, 5, 7, 9, 9, 9, 9, 11]
```

2c

Benchmark the algorithm efficiency (comparison of merge-sort with insertion sort) with TimeIt module

```
import random
import timeit


# MERGE SORT
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    return merge(merge_sort(arr[:mid]), merge_sort(arr[mid:]))


def merge(left, right):
    result = []
    i = j = 0
```

```
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result


# INSERTION SORT
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

    return arr


# BENCHMARK
sizes = [100, 1000, 5000]

for size in sizes:
    data = [random.randint(1, 10000) for _ in range(size)]

    insertion_time = timeit.timeit(
        lambda: insertion_sort(data[:]), number=5
    )

    merge_time = timeit.timeit(
```

```
        lambda: merge_sort(data[:]), number=5
    )

    print(f"\nSize: {size}")
    print(f"Insertion Sort: {insertion_time:.6f} sec")
    print(f"Merge Sort:     {merge_time:.6f} sec")
_


Size: 100
Insertion Sort: 0.000573 sec
Merge Sort:     0.000606 sec

Size: 1000
Insertion Sort: 0.111590 sec
Merge Sort:     0.008707 sec

Size: 5000
Insertion Sort: 2.934398 sec
Merge Sort:     0.059630 sec
```

Task 3: Tower of Hanoi problem – Application of the recursion algorithm

3a

Test the following Python script for the Tower of Hanoi problem for different disk sizes and report the results for atleast 2 different disk sizes.

```
Enter number of disks: 2
The moves are:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B

_


Enter number of disks: 4
```

```
The moves are:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Move disk 4 from A to B
Move disk 1 from C to B
Move disk 2 from C to A
Move disk 1 from B to A
Move disk 3 from C to B
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
```

3b

Modify the code for TowerOfHanoi.py, so that the program finds the number of moves needed to move n disks from tower A to tower B.

```python
move_count = 0

def moveDisks(n, fromTower, toTower, auxTower):
    global move_count
    if n == 1:
        print(f"Move disk 1 from {fromTower} to {toTower}")
        move_count += 1
    else:
        moveDisks(n - 1, fromTower, auxTower, toTower)
        print(f"Move disk {n} from {fromTower} to {toTower}")
        move_count += 1
        moveDisks(n - 1, auxTower, toTower, fromTower)


def main():
    global move_count
    n = int(input("Enter number of disks: "))
```

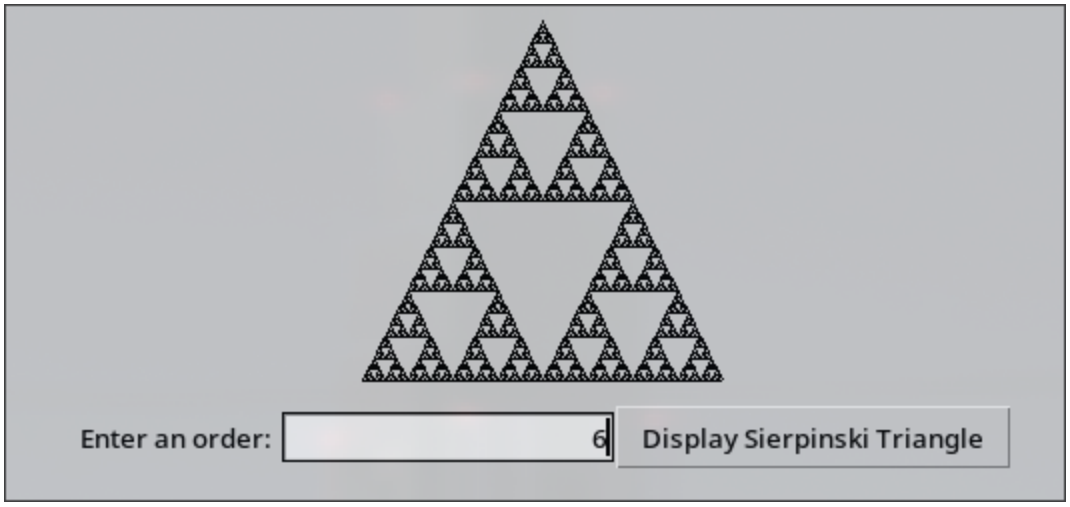
```
    move_count = 0
    moveDisks(n, 'A', 'B', 'C')
    print(f"\nTotal moves: {move_count}")


main()
```

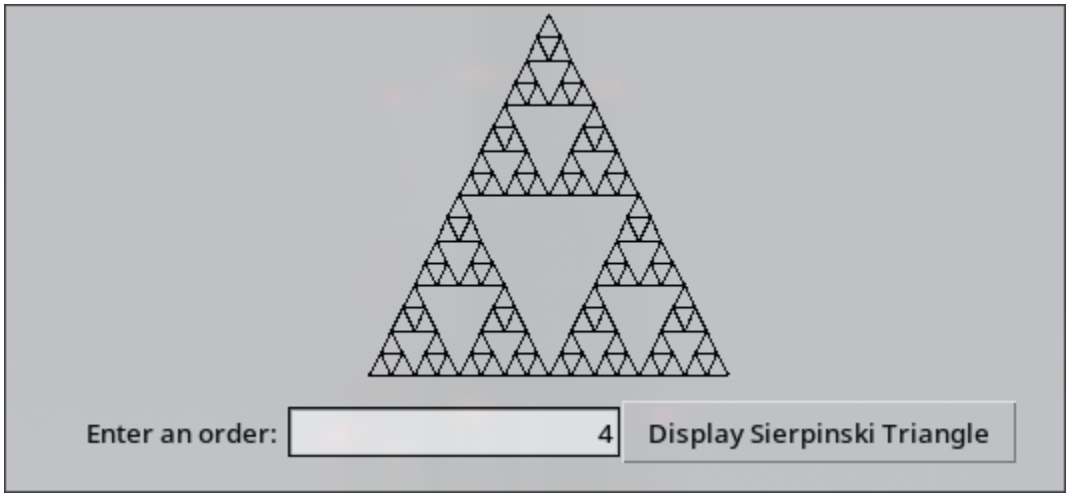
```
--

Enter number of disks: 3
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C
Move disk 3 from A to B
Move disk 1 from C to A
Move disk 2 from C to B
Move disk 1 from A to B

Total moves: 7

--

Enter number of disks: 4
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Move disk 4 from A to B
Move disk 1 from C to B
Move disk 2 from C to A
Move disk 1 from B to A
Move disk 3 from C to B
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B

Total moves: 15
```

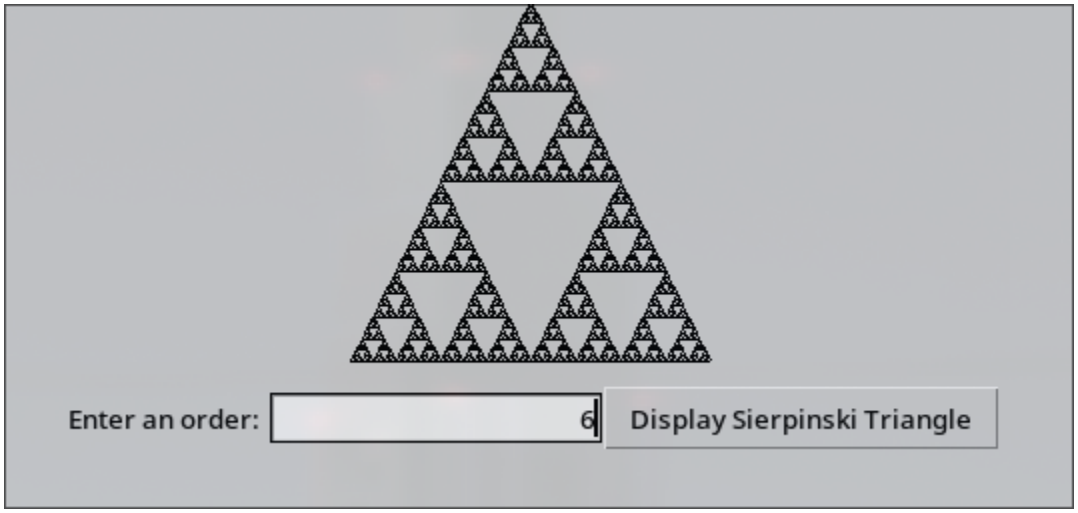
Task 4: Sierpinski Triangle – Fractals Computation using recursion algorithm

4a

You need to run the TkInter code sierpinski_triangle.py for this simple fractal problem and test it for atleast 3 different orders other than 0,1,2,and 3, and report the results


Enter an order:
6
Display Sierpinski Triangle


Enter an order:
4
Display Sierpinski Triangle

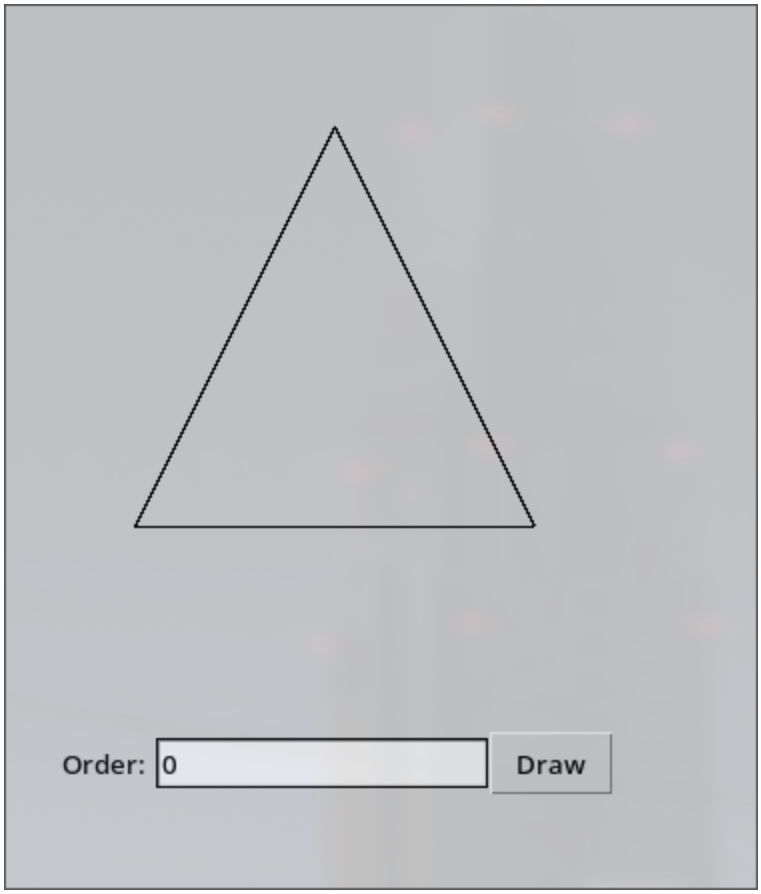

Enter an order:
6
Display Sierpinski Triangle

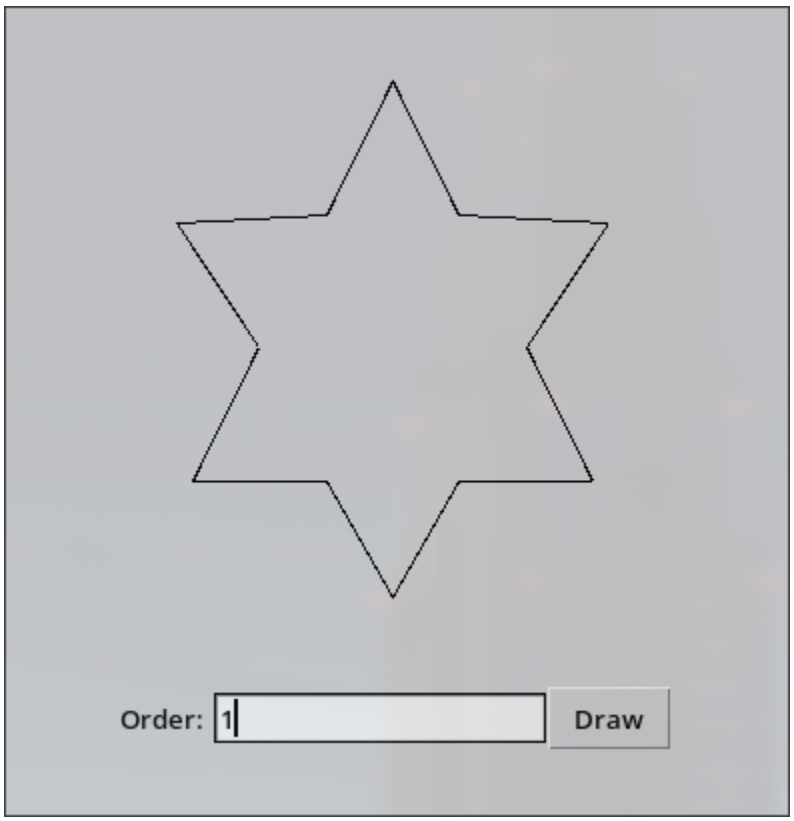
4b

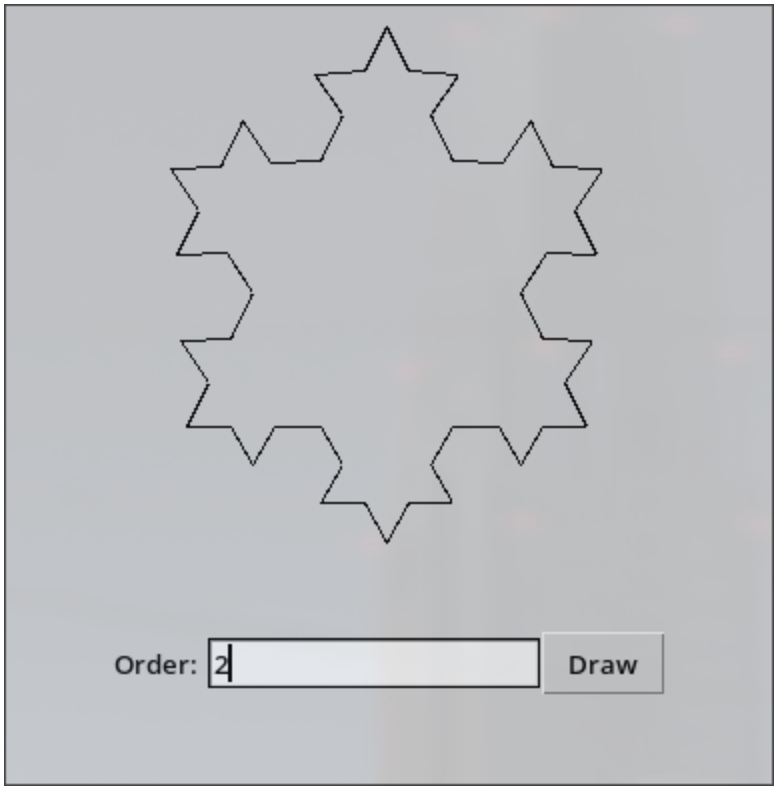
Now, you will modify the sierpinski_triangle.py program to display the snowflake fractalcalled the Koch snowflake

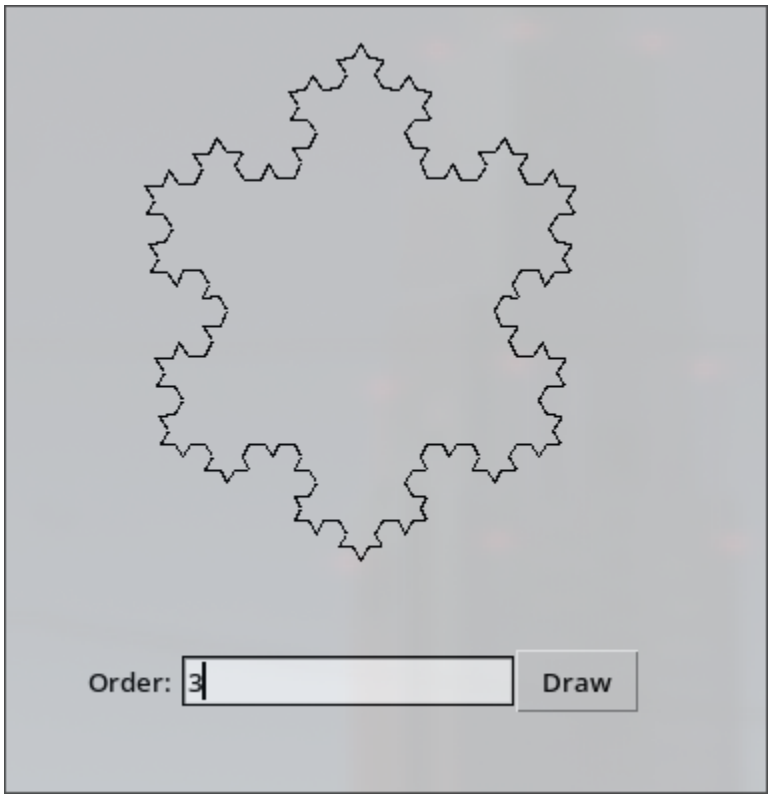
```
from tkinter import *
import math

class KochSnowflake:
   def __init__(self):
       window = Tk()
       window.title("Koch Snowflake")

       self.width = 400
       self.height = 400

       self.canvas = Canvas(window, width=self.width, height=self.height)
       self.canvas.pack()

       frame = Frame(window)
       frame.pack()

       Label(frame, text="Order: ").pack(side=LEFT)
       self.order = StringVar()
       Entry(frame, textvariable=self.order).pack(side=LEFT)

       Button(frame, text="Draw", command=self.draw).pack(side=LEFT)

       window.mainloop()

   def draw(self):
       self.canvas.delete("line")

       p1 = (100, 300)
       p2 = (300, 300)
       p3 = (200, 100)

       order = int(self.order.get())

       self.koch(p1, p2, order)
       self.koch(p2, p3, order)
       self.koch(p3, p1, order)
```

```
   def koch(self, p1, p2, order):
       if order == 0:
           self.canvas.create_line(p1[0], p1[1], p2[0], p2[1],
tags="line")
       else:
           x1, y1 = p1
           x2, y2 = p2

           dx = (x2 - x1) / 3
           dy = (y2 - y1) / 3

           pA = (x1 + dx, y1 + dy)
           pB = (x1 + 2*dx, y1 + 2*dy)

           # Peak of triangle
           angle = math.radians(60)
           px = pA[0] + (dx * math.cos(angle) - dy * math.sin(angle))
           py = pA[1] + (dx * math.sin(angle) + dy * math.cos(angle))
           pC = (px, py)

           self.koch(p1, pA, order-1)
           self.koch(pA, pC, order-1)
           self.koch(pC, pB, order-1)
           self.koch(pB, p2, order-1)


KochSnowflake()
```


Order:
0
Draw


Order:
1
Draw


Order: 2
Draw


Order: 3
Draw